Day 2 – Participant Setup and Resource Provisioning

Objective: Each participant independently creates the Azure environment required for all three Day 2 labs.

1. Verify Azure access

Open Azure Portal and sign in using your assigned account.

Verify:

- Correct Microsoft Entra tenant.
- Correct Azure subscription.
- Access to the shared training Resource Group.
- Permission to create resources in that Resource Group.

For the exercises, participants should have Contributor access at the Resource Group scope.

Important permission distinction:

- Contributor can create and manage resources.
- Contributor cannot assign Azure RBAC roles.
- Role assignments require an appropriate role such as Owner, User Access Administrator or Role Based Access Control Administrator.

For Day 2, we can use the Storage Account connection configuration created with the Function App, avoiding an additional managed-identity role assignment exercise.

2. Create the Day 2 Storage Account

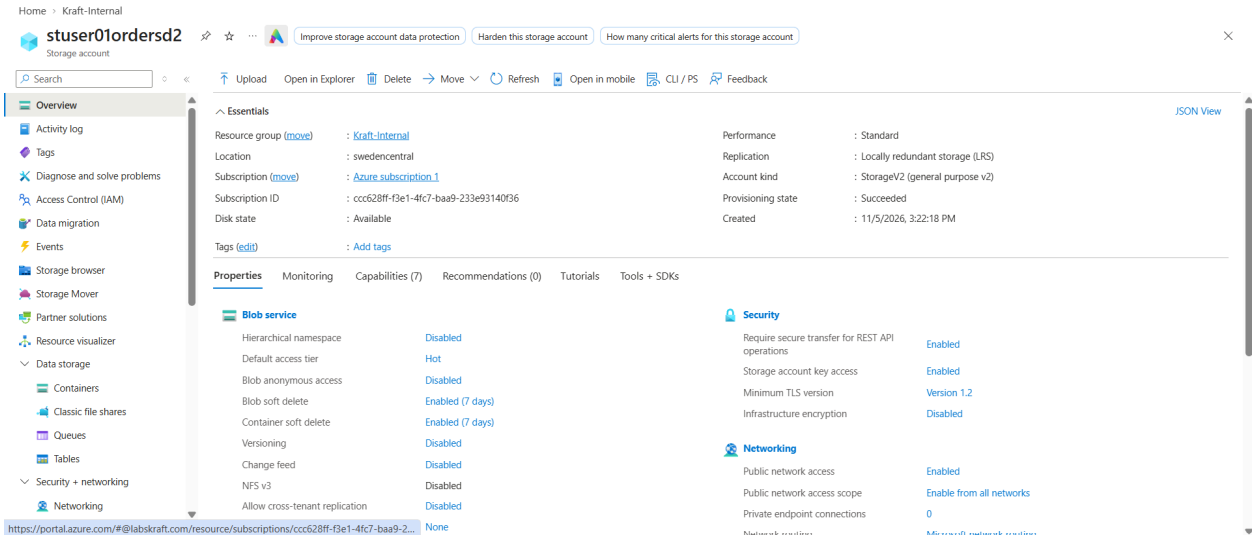
1. Search for Storage accounts in Azure Portal.
2. Select Create.
3. Select your existing shared Resource Group.
4. Enter the name:

```
stuser01ordersd2
```

Replace `user01` with your assigned participant ID.

5. Region: Select the approved training region.
6. Performance: Standard.
7. Redundancy: Locally-redundant storage (LRS).
8. Account kind: StorageV2 (general-purpose v2).
9. Keep other settings at their default training values.
10. Select Review + create → Create.

Verify that deployment succeeds.



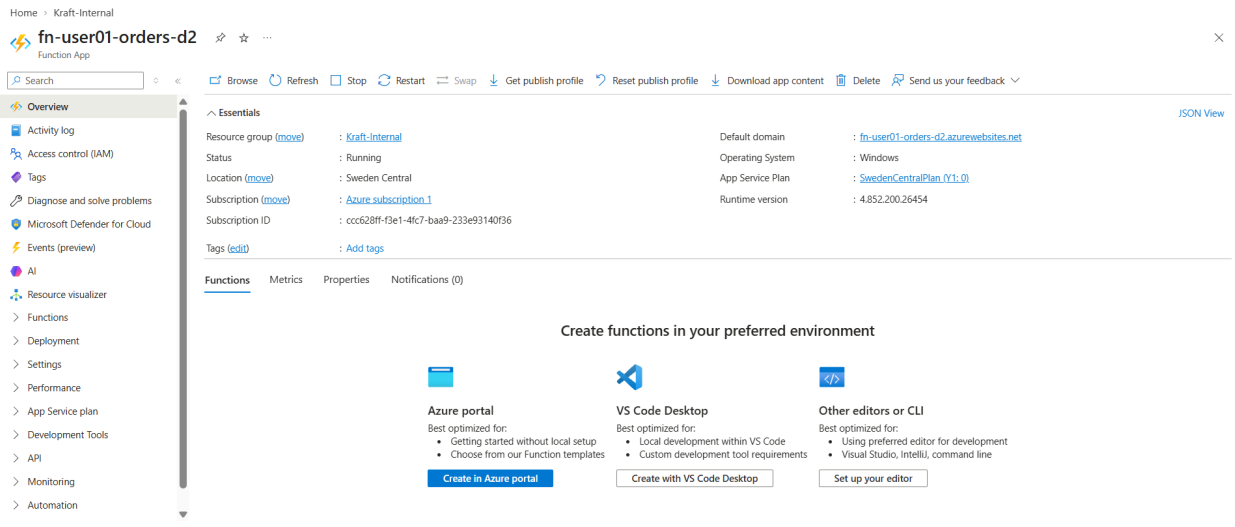
3. Create the Function App

1. Search for Function App.
2. Select Create.
3. Select the same Resource Group.
4. Enter:

```
None
fn-user01-orders-d2
```

5. Select the approved region.
6. Hosting plan: Flex Consumption.
7. Operating system: Linux.
8. Runtime: .NET.
9. Version: .NET 10 isolated worker.
10. Select the Storage Account created in Step 2.
11. Enable Application Insights.
12. Review and create.

Wait for deployment to complete.



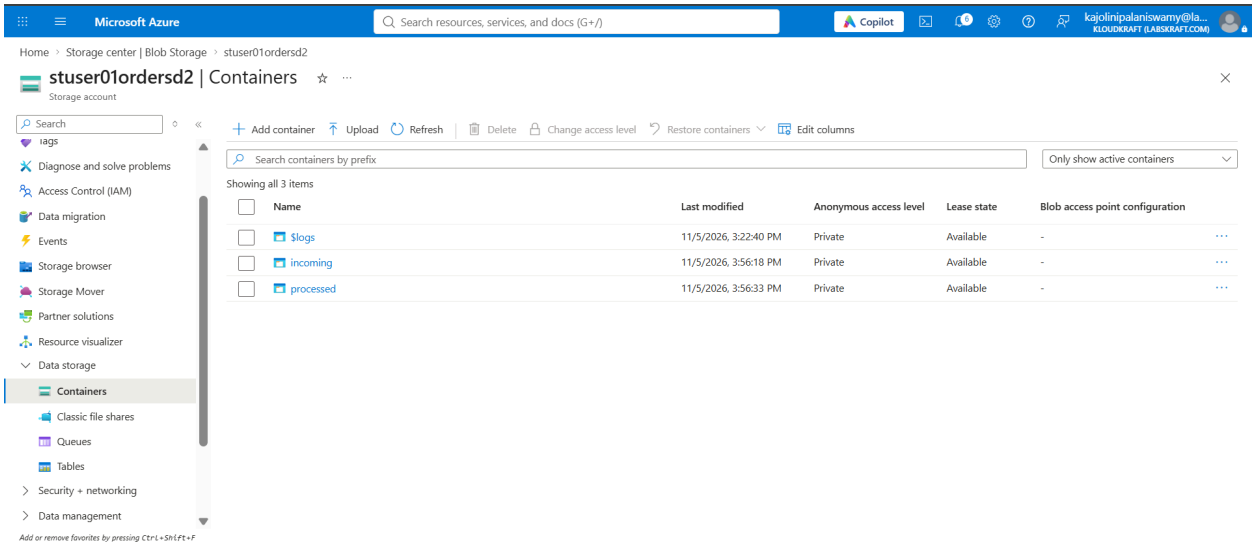
4. Create the Blob containers

Open your Storage Account.

Navigate to Data storage → Containers.

Create these two containers:

```
None
incoming
processed
```

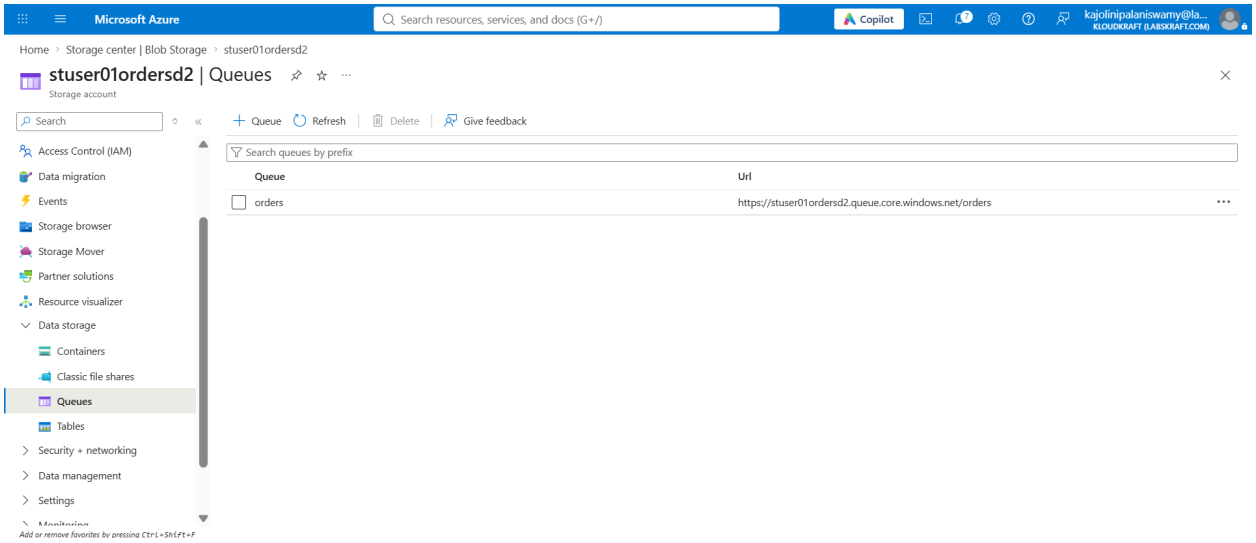


5. Create the Queue

Navigate to Data storage → Queues.

Create:

```
None
orders
```



These Azure resources will be used in Lab 2.3.

6. Verify Function App configuration

Open your Function App.

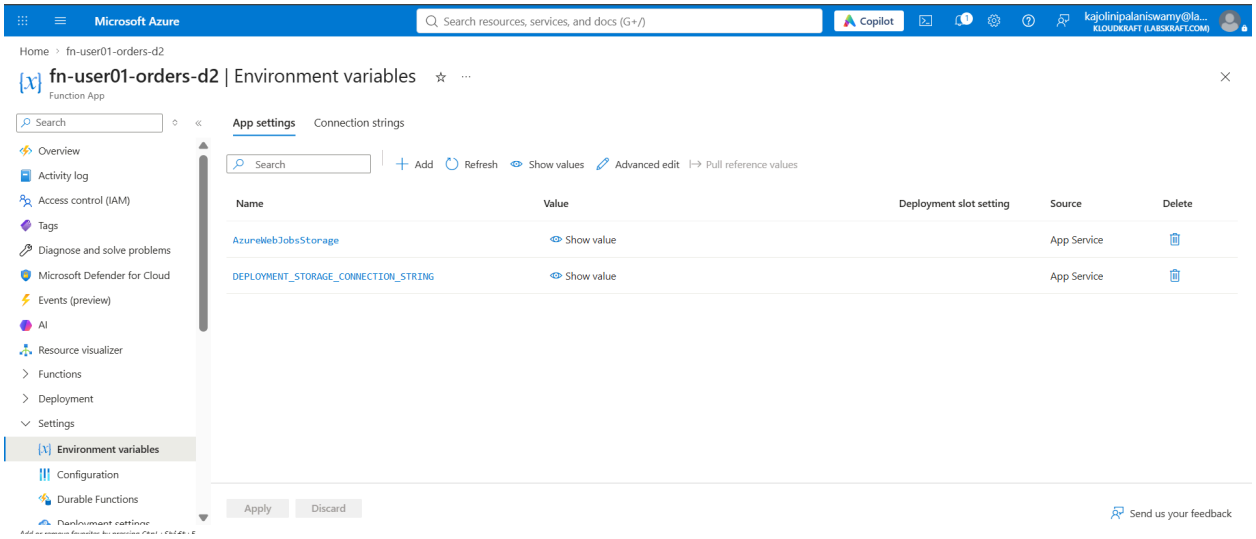
Check:

- Hosting plan: Flex Consumption.
- OS: Linux.
- Runtime: .NET 10.
- State: Running.
- Application Insights: Connected.

Open Settings → Environment variables.

Verify that `AzureWebJobsStorage` is configured for the Storage Account created in Step 2.

Do not expose connection strings or account keys in screenshots.



7. Verify local development tools

Open VS Code terminal:

```
None
dotnet --version
func --version
az --version
```

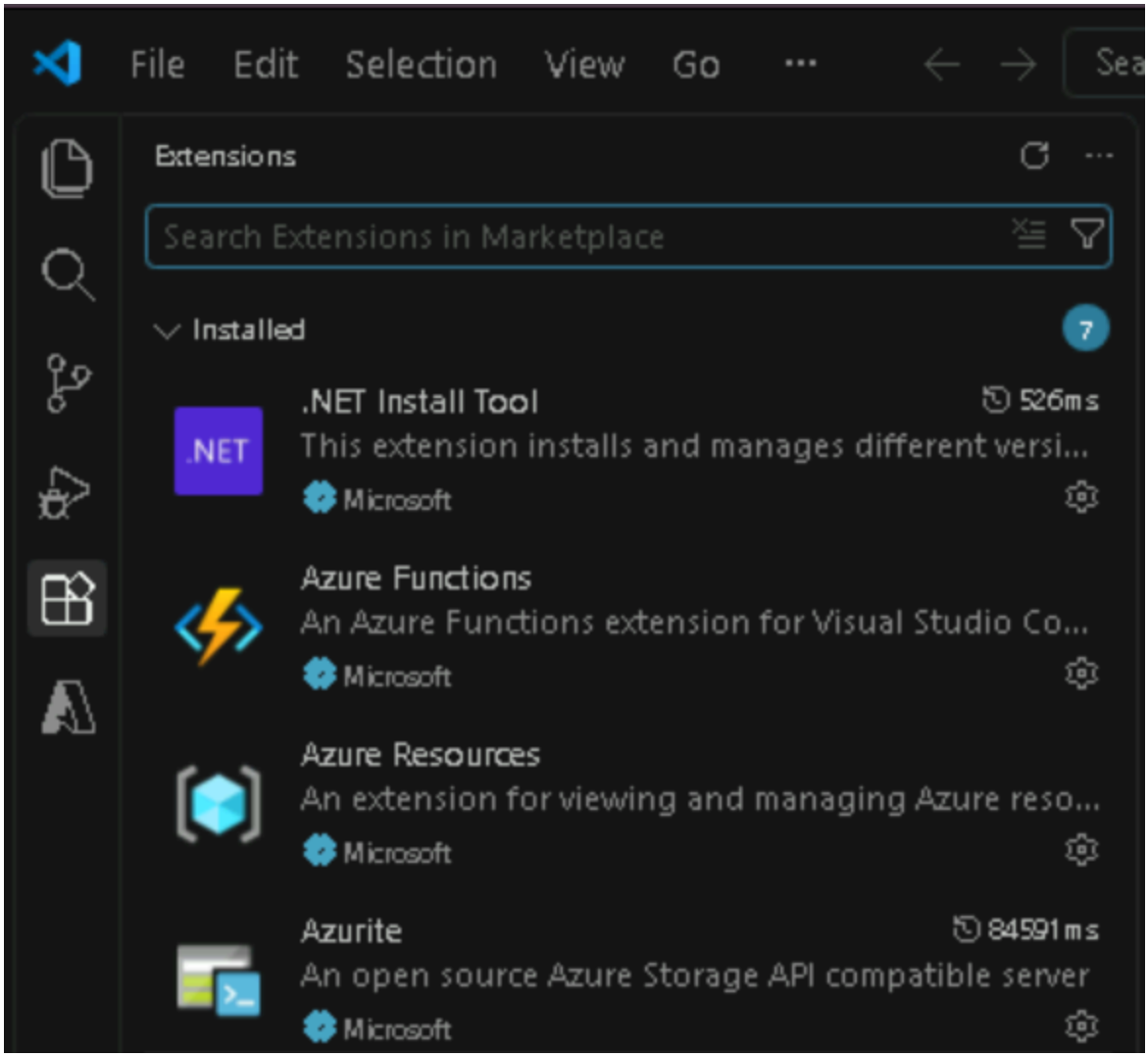
Expected:

- .NET SDK 10.x
- Azure Functions Core Tools 4.x
- Azure CLI installed



Confirm these VS Code extensions:

- Azure Functions
- Azure Resources
- Azurite



Ensure the existing `contoso-fn` project and `orders.csv` training asset are available.

```
PS C:\Users\LabsKraft> Get-Item orders.csv


    Directory: C:\Users\LabsKraft


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         11/5/2026   2:22 PM           1016 orders.csv
```

```
PS C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders> dotnet add package Microsoft.Azure.Functions.
sions.Timer
info : X.509 certificate chain validation will use the default trust store selected by .NET for code signing.
info : X.509 certificate chain validation will use the default trust store selected by .NET for timestamping.
info : Adding PackageReference for package 'Microsoft.Azure.Functions.Worker.Extensions.Timer' into project 'C:
raft\Day02_Azure_Functions\day02-contoso-orders\day02-contoso-orders.csproj'.
info :   GET https://api.nuget.org/v3/registration5-gz-semver2/microsoft.azure.functions.worker.extensions.time
info :   OK https://api.nuget.org/v3/registration5-gz-semver2/microsoft.azure.functions.worker.extensions.timer
419ms
info : Restoring packages for C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\day02-contoso-order
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.timer/index.jso
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.timer/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.timer/4.3.1/mic
.functions.worker.extensions.timer.4.3.1.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.timer/4.3.1/micr
functions.worker.extensions.timer.4.3.1.nupkg 201ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.core/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.core/index.json 195ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.core/1.18.0/microsoft.azur
worker.core.1.18.0.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.core/1.18.0/microsoft.azure
orker.core.1.18.0.nupkg 203ms
info : Installed Microsoft.Azure.Functions.Worker.Extensions.Timer 4.3.1 from https://api.nuget.org/v3/index.js
rs\LabsKraft\.nuget\packages\microsoft.azure.functions.worker.extensions.timer\4.3.1 with content hash uNKWSuel
```

```
713yBkXZCkYnF/OcssX1ONY+vM+AP+rm/wPR7iJ5xeDgOp5sOCKHEu5XYsbhZsfCRg==.
info : Installed Microsoft.Azure.Functions.Worker.Core 1.18.0 from https://api.nuget.org/v3/index.json to C:\Us
t\.nuget\packages\microsoft.azure.functions.worker.core\1.18.0 with content hash AFHJMjPB7JhwnWnGFbroSGMRa5ertG
3YmjrM29BasD9zL                                                                    /cKdVKQ==.
info :   CACHE  Follow link (ctrl + click)  v3/vulnerabilities/index.json
info :   CACHE https://api.nuget.org/v3-vulnerabilities/2026.11.04.11.49.31/vulnerability.base.json
info :   CACHE https://api.nuget.org/v3-vulnerabilities/2026.11.04.11.49.31/2026.11.05.11.49.38/vulnerability.u
info : Package 'Microsoft.Azure.Functions.Worker.Extensions.Timer' is compatible with all the specified framewo
ct 'C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\day02-contoso-orders.csproj'.
info : PackageReference for package 'Microsoft.Azure.Functions.Worker.Extensions.Timer' version '4.3.1' added t
sers\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\day02-contoso-orders.csproj'.
info : Writing assets file to disk. Path: C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\obj\pro
json
log  : Restored C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\day02-contoso-orders.csproj (in 2
PS C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders> dotnet restore
Restore complete (1.6s)

Build succeeded in 3.3s
PS C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders>
```

8. Resource readiness checklist

Resource/check	Expected

Azure subscription	Accessible
Shared Resource Group	Accessible
Storage Account	Created
Function App	Created
Hosting plan	Flex Consumption
Runtime	.NET 10 isolated
Application Insights	Enabled
Blob container	`incoming`
Blob container	`processed`
Queue	`orders`
Local project	Available

Training dataset	Available

9. Naming convention

Use your participant ID consistently.

Resource	Example
Function App	`fn-user01-orders-d2`
Storage Account	`stuser01ordersd2`
Application Insights	`appi-user01-orders-d2`
Blob container	`incoming`
Blob container	`processed`
Queue	`orders`

10. End-of-day cleanup

After completing Labs 2.1, 2.2 and 2.3:

1. Open the shared Resource Group.

2. Identify your Day 2 resources.
3. Delete your Function App.
4. Delete your Day 2 Storage Account.
5. Delete your dedicated Application Insights resource, if separately provisioned.
6. Delete any dedicated Log Analytics workspace created exclusively for your lab.
7. Verify that your resources have been deleted.

Do not delete the shared Resource Group or resources belonging to other participants.

One important training rule going forward: Every day's setup guide will contain the complete participant-side resource creation, configuration, verification and cleanup steps. No administrator intervention will be assumed.

For Lab 2.1, participants first complete this setup, then follow the Timer Function development and deployment guide.